PROGRAMMING IN JAVA

ASSIGNMENT 3 – CO3

Topic: Exception Handling and Multithreading in Java

INTRODUCTION

Java is an object-oriented programming language widely used for developing reliable, secure and efficient software applications. Exception handling and multithreading are two important concepts that help developers create robust applications.

Exception handling is a mechanism used to identify and manage errors that occur during program execution. Java provides several keywords, including try, catch, finally, throw and throws, to handle exceptions effectively.

Multithreading is the process of executing multiple threads concurrently within a single application. It improves application responsiveness and allows different tasks to execute independently.

This assignment focuses on the practical implementation and analysis of exception handling and multithreading concepts through real-world applications such as ATM transactions, online examination portals, sales report generation, examination scanning, cinema seat booking, hostel wallet transfers and semester result processing.

OBJECTIVES

1. To understand the Java exception hierarchy.
2. To differentiate checked and unchecked exceptions.
3. To understand try, catch, finally, throw and throws.
4. To implement user-defined exceptions and exception chaining.
5. To understand resource management using try-with-resources.
6. To study thread creation and the thread life cycle.
7. To understand race conditions and synchronization.
8. To identify and prevent deadlocks.
9. To implement inter-thread communication.
10. To understand ExecutorService, Callable, Future and concurrent collections.
11. To analyse the time and space complexity of Java applications.

Q1 – HANDLING FAILURE IN A BANK ATM SIMULATION

1.1 Introduction

An Automated Teller Machine (ATM) is an electronic banking system that allows customers to withdraw money from their bank accounts. During a withdrawal transaction, different types of errors may occur.

For example, a customer may enter letters instead of numbers, provide a negative withdrawal amount, request an amount greater than the available balance or encounter a missing card-reader file.

These situations must be handled properly to prevent the application from terminating unexpectedly.

Java exception handling provides a mechanism to identify errors and display appropriate messages to users.

1.2 Failure Identification

The ATM withdrawal process may fail at different stages.

1. Invalid account number input.
2. Invalid withdrawal amount.
3. Negative withdrawal amount.
4. Withdrawal amount exceeding the account balance.
5. Missing card-reader file.
6. Serious system-level errors.

These failures can be classified into checked exceptions, unchecked exceptions and errors.

1.3 Exception Classification

Failure	Exception type	Classification
Missing card-reader file	IOException	Checked
Invalid numeric input	NumberFormatException	Unchecked
Negative withdrawal amount	IllegalArgumentException	Unchecked
Insufficient balance	User-defined exception	Checked
Memory-related system failure	OutOfMemoryError	Error

1.4 Exception Handling Mechanism

Java provides the following keywords for exception handling:

- **try:** Contains statements that may generate exceptions.
- **catch:** Handles exceptions generated inside the try block.
- **finally:** Executes after the try/catch processing in normal completion and exception-handling paths.
- **throw:** Explicitly generates an exception.
- **throws:** Declares exceptions that a method may pass to its caller.

Multi-catch allows multiple exception types to be handled using a single catch block.

1.5 Student A – Exception Design

Student A is responsible for designing the exception hierarchy and explaining the exception-handling mechanism.

The Throwable hierarchy contains two major branches: Error and Exception.

Error represents serious system-level problems, whereas Exception represents conditions that applications may handle.

The exception hierarchy relevant to the ATM application includes IOException, RuntimeException, NumberFormatException and custom exceptions.

Justification for catch block order:

Specific exceptions should be handled before general exceptions. If a general Exception catch block is placed before a specific catch block, the specific block may become unreachable.

Importance of finally:

The finally block is used for cleanup operations, such as closing resources and displaying transaction completion messages.

1.6 Student B – Testing and Tracing

Student B is responsible for designing test cases and analysing the execution behaviour.

Test case	Input condition	Expected behaviour
1	Valid withdrawal amount	Transaction successful
2	Alphabetic input	Number format error

3	Negative amount	Invalid amount message
4	Amount greater than balance	Insufficient balance message
5	Missing card-reader file	File error message

A stack trace provides information about the exception type, error message and sequence of method calls that led to the failure.

Each line of a stack trace helps identify the location where the exception occurred and the methods involved in the execution.

If a broader catch block is placed before a narrower catch block, the narrower catch block becomes unreachable.

1.7 Joint Design Question

Should the ATM program catch every exception using a single catch(Exception e)?

Using a single catch(Exception e) is acceptable in certain application-level error-handling situations where unexpected exceptions must be logged and prevented from terminating the application.

However, it is not recommended for every situation because different exceptions require different handling methods.

Using specific catch blocks improves debugging, readability and error identification.

1.8 Alternative Concept and Complexity Analysis

The ATM application can also be implemented without exceptions by validating every input before performing the withdrawal operation.

Methods such as Scanner.hasNextInt() and Scanner.hasNextDouble() can be used to validate user input.

The program can return error codes such as -1 for invalid input and -2 for insufficient balance.

Factor	**Exception approach**	**Error-code approach**
Error identification	Catch blocks	Return codes
Readability	Suitable for exceptional conditions	Suitable for frequent validation
Processing cost	Higher when exceptions are thrown	Generally lower
Time complexity	O(1) for basic transaction processing	O(1)

Additional memory	Exception objects and stack traces	O(1)

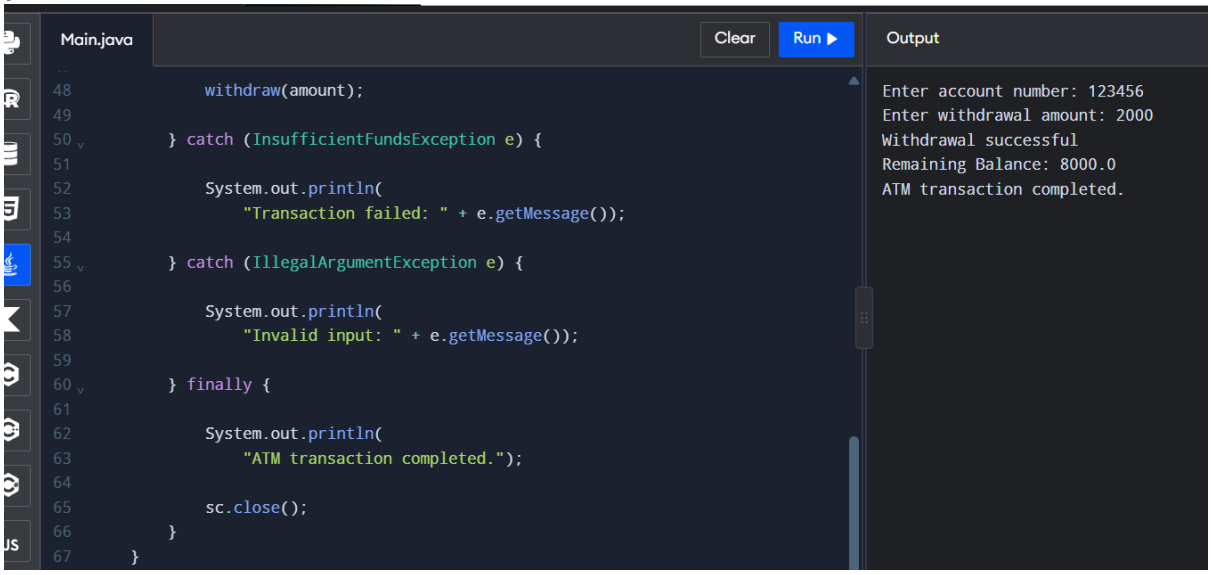
Exception creation and stack-trace generation require additional processing. Error-code handling is generally more efficient when validation failures occur frequently.

PROGRAM:

```
import java.util.*;
class InsufficientFundsException extends Exception {

   InsufficientFundsException(String message) {
      super(message);
   }
}
public class Main {
   static double balance = 10000;
   public static void withdraw(String amountText)
         throws InsufficientFundsException {
      double amount = Double.parseDouble(amountText);
      if (amount <= 0) {
         throw new IllegalArgumentException(
            "Withdrawal amount must be positive");
      }
      if (amount > balance) {
         throw new InsufficientFundsException(
            "Insufficient account balance");
      }
      balance = balance - amount;
      System.out.println("Withdrawal successful");
      System.out.println("Remaining Balance: " + balance);
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      try {
         System.out.print("Enter account number: ");
         String account = sc.nextLine();
         System.out.print("Enter withdrawal amount: ");
         String amount = sc.nextLine();
         withdraw(amount);
      } catch (InsufficientFundsException e) {
         System.out.println(
            "Transaction failed: " + e.getMessage());
      } catch (IllegalArgumentException e) {
         System.out.println(
            "Invalid input: " + e.getMessage());
      } finally {
         System.out.println(
            "ATM transaction completed.");
```

```
        sc.close();
    }
  }
}
```



1.9 Conclusion

The ATM simulation demonstrates the importance of exception handling in banking applications. Using checked exceptions, unchecked exceptions, multi-catch and finally improves program reliability and helps manage unexpected failures.

Q2 – USER-DEFINED EXCEPTIONS AND PROPAGATION IN AN ONLINE EXAMINATION PORTAL

2.1 Introduction

An online examination portal allows students to complete examinations and upload their answer files through an online platform.

During submission, different failures may occur. The examination window may close, the student may not be registered for the course, or the uploaded answer file may exceed the permitted size.

These errors are checked at different application layers, including the controller, service and storage layers.

Java supports user-defined exceptions that allow developers to create meaningful exception classes for application-specific situations.

2.2 Objectives

1. To understand user-defined exceptions.
2. To differentiate checked and unchecked exceptions.
3. To implement exception propagation.
4. To understand throw and throws.
5. To implement exception chaining.
6. To preserve the original cause of an exception.

2.3 User-Defined Exception Hierarchy

The online examination portal requires a hierarchy of custom exceptions.

The proposed hierarchy contains:

- ExamException
- ExamClosedException
- StudentNotRegisteredException
- SubmissionStorageException

ExamException acts as the parent exception class.

The remaining exceptions represent specific examination-related failures.

SubmissionStorageException represents errors occurring during answer-file storage.

2.4 Checked and Unchecked Exceptions

Checked exceptions are verified by the Java compiler. The programmer must handle them or declare them using the throws keyword.

Unchecked exceptions occur during runtime and do not require mandatory handling or declaration.

For the online examination portal, examination closure, registration failure and storage failure are suitable checked exceptions because they represent expected business or operational conditions that should be handled explicitly.

2.5 Exception Propagation

Exception propagation is the process through which an exception moves from one method to its calling method until it is handled.

In the online examination portal, the storage layer may generate an exception while saving an answer file.

The exception is passed to the service layer and then to the controller layer.

The controller finally handles the exception and displays an appropriate message to the student.

Propagation flow:

Storage Layer → Service Layer → Controller Layer → Error Message to Student.

2.6 Student A – Exception Hierarchy and Propagation Design

Student A is responsible for designing the exception hierarchy and explaining the propagation process.

The exception hierarchy separates examination-related failures into meaningful categories.

Reasons for using checked exceptions:

1. ExamClosedException ensures that the examination deadline is respected.
2. StudentNotRegisteredException ensures that only registered students can submit answers.
3. SubmissionStorageException identifies problems during file storage.

The controller layer is responsible for handling the propagated exception and displaying a user-friendly error message.

2.7 Student B – Failure Scenarios and Chaining Evidence

Student B is responsible for constructing failure scenarios and demonstrating exception chaining.

Scenario	**Exception**	**Expected message**
Examination time expired	ExamClosedException	Examination has ended
Student not registered	StudentNotRegisteredException	Student is not registered
File size exceeds limit	SubmissionStorageException	Could not store answer file

Exception chaining:

Exception chaining is the process of associating one exception with another exception that originally caused the failure.

The getCause() method retrieves the original cause.

For example, when an IOException occurs during file storage, it can be wrapped inside SubmissionStorageException.

If the original cause is not passed to the constructor, getCause() returns null and the original technical error information is lost.

2.8 Joint Conclusion

It is better for the storage layer to generate a meaningful SubmissionStorageException rather than allowing an IOException to travel unchanged to the user interface.

This provides better abstraction, improves debugging and prevents internal storage implementation details from being directly exposed to users.

2.9 Alternative Concept and Complexity Analysis

If custom exceptions extend RuntimeException instead of Exception, they become unchecked exceptions.

The major changes are:

1. The compiler does not require mandatory exception handling.
2. The throws declaration becomes optional.
3. Calling methods can invoke the method without mandatory try-catch blocks.
4. The programmer must ensure that errors are handled appropriately.

Factor	**Checked exception**	**Unchecked exception**
Compiler checking	Mandatory handling or declaration	Not mandatory
Method declaration	Usually uses throws	Optional
Error handling	Explicit	Optional
Stack trace	Captured	Captured
Stack-trace complexity	O(d)	O(d)

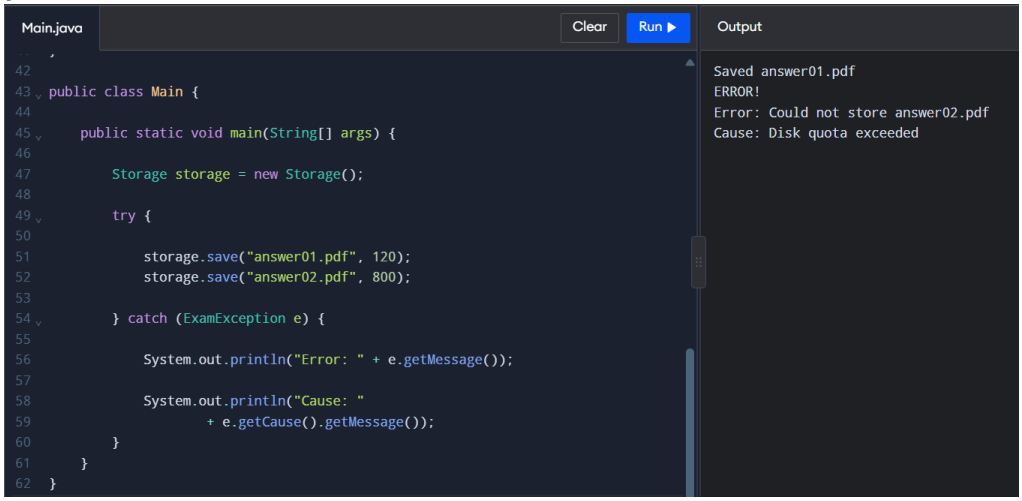
Here, d represents the stack depth.

Exception chaining maintains a reference to the original exception. Therefore, both exception objects and their associated stack-trace information may remain in memory while they are reachable.

PROGRAM:

```
import java.io.*;
class ExamException extends Exception {
   ExamException(String message) {
      super(message);
   }
```

```
    ExamException(String message, Throwable cause) {
        super(message, cause);
    }
}
class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String message, Throwable cause) {
        super(message, cause);
    }
}
class Storage {
    void save(String fileName, long sizeKb)
            throws SubmissionStorageException {
        try {
            if (sizeKb > 500) {
                throw new IOException("Disk quota exceeded");
            }
            System.out.println("Saved " + fileName);
        } catch (IOException e) {
            throw new SubmissionStorageException(
                "Could not store " + fileName, e);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Storage storage = new Storage();
        try {
            storage.save("answer01.pdf", 120);
            storage.save("answer02.pdf", 800);
        } catch (ExamException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Cause: "
                    + e.getCause().getMessage());
        }
    }
}
```



2.10 Conclusion

The online examination portal demonstrates how user-defined exceptions, exception propagation and exception chaining can improve application reliability. These concepts help identify errors at different application layers and preserve useful information for debugging.

Q3 – RESOURCE SAFETY, DEBUGGING AND ROBUST CODE

3.1 Introduction

A retail company generates daily sales reports by reading data from several store files and writing a summary into an output file.

During execution, the application may encounter errors such as missing files, corrupted data or file-access problems.

If resources are not closed properly, the program may experience resource leakage and errors such as Too many open files.

Java provides two methods for resource management: manual resource closing using finally and automatic resource closing using try-with-resources.

3.2 Objectives

1. To understand resource management.
2. To identify resource leakage.
3. To compare manual resource closing and try-with-resources.
4. To understand debugging using breakpoints.
5. To understand suppressed exceptions.
6. To improve application reliability.

3.3 Resource Management

A resource is an external object required by an application during execution.

Examples include:

- File readers.
- File writers.
- Database connections.
- Network connections.
- Input and output streams.

These resources must be closed after their use to avoid unnecessary resource consumption.

3.4 Manual Resource Closing

Manual resource management uses try, catch and finally blocks.

The programmer is responsible for opening and closing every resource.

The finally block is used to close the resources even when an exception occurs.

However, manual closing requires careful implementation. If an exception occurs during cleanup, another resource may remain open unless closing operations are handled independently.

3.5 Try-with-Resources

Try-with-resources is a Java feature that automatically closes resources declared inside the try statement.

It reduces the amount of code required and makes resource management more reliable.

Resources are closed automatically in the reverse order of their declaration.

For example, if a reader is declared first and a writer is declared second, the writer is closed first, followed by the reader.

3.6 Comparison of Resource Management Approaches

Feature	**Manual closing**	**Try-with-resources**
Closing operation	Programmer responsibility	Automatic
Code length	Longer	Shorter
Resource leakage risk	Higher	Lower
Error handling	More complicated	Easier
Readability	Less readable	More readable
Reliability	Depends on implementation	More reliable

3.7 Student A – Debugging and Experimental Analysis

Student A is responsible for debugging the manual resource-management implementation.

The debugging process includes:

1. Opening the program in an IDE.
2. Placing breakpoints near resource-opening statements.

3. Executing the application step by step.
4. Observing the creation of reader and writer objects.
5. Introducing invalid data into the input file.
6. Checking whether all resources are closed after failure.

A failure can be created by inserting alphabetic data into a file that should contain numeric sales values.

When the program attempts to convert the invalid data into an integer, a NumberFormatException occurs.

The manual implementation must execute its cleanup logic, while try-with-resources automatically attempts to close initialized resources.

3.8 Student B – Design and Correctness

Student B is responsible for identifying resources and explaining the correctness of the resource-management approaches.

The main resources used in the report generator are:

1. BufferedReader for reading store data.
2. BufferedWriter for writing the summary report.

Both resources must be closed after processing.

Suppressed exceptions:

A suppressed exception occurs when an exception is generated during resource closing while another exception is already being thrown.

The original exception remains the primary exception, and the closing exception is attached as a suppressed exception.

3.9 Joint Engineering Decision

Try-with-resources improves resource safety, but it does not automatically guarantee complete application correctness.

Additional error handling is required for:

- Corrupted input files.
- Invalid sales information.
- Disk-space problems.
- Network failures.

- Concurrent access to files.
- Incomplete report generation.

Therefore, validation, logging, exception handling and backup mechanisms should also be included.

3.10 Alternative Concept and Complexity Analysis

The report generator can also be implemented using explicit close() calls inside a finally block.

Another possible approach is to avoid closing resources, but this is not recommended because file handles may accumulate.

Let f represent the number of store files.

Approach	Time complexity	Maximum open resources
Manual closing	O(f)	Depends on cleanup implementation
Try-with-resources	O(f)	Depends on resource scope
No closing	O(f) processing	Can grow to O(f)

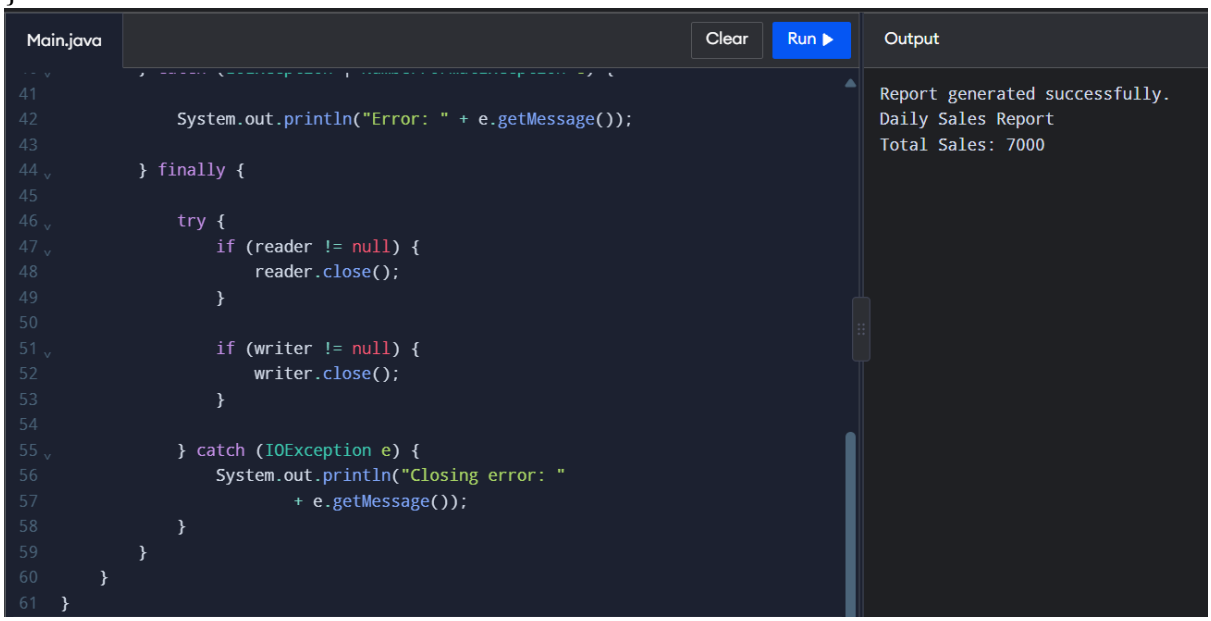
If files are processed one at a time and closed immediately, the number of simultaneously open file handles is O(1).

If all files are opened together without closing, the number of open file handles may grow to O(f).

PROGRAM:

```
import java.io.*;
public class Main {
   public static void main(String[] args) {
      BufferedReader reader = null;
      BufferedWriter writer = null;
      try {
         String data = "1000\n2000\n1500\n2500";
         reader = new BufferedReader(
            new StringReader(data));
         StringWriter output = new StringWriter();
         writer = new BufferedWriter(output);
         String line;
         int total = 0;
         while ((line = reader.readLine()) != null) {
            total += Integer.parseInt(line);
         }
```

```
            writer.write("Daily Sales Report");
            writer.newLine();
            writer.write("Total Sales: " + total);
            writer.flush();
            System.out.println(
               "Report generated successfully.");
            System.out.println(output.toString());
        } catch (IOException | NumberFormatException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            try {
                if (reader != null) {
                    reader.close();
                }
                if (writer != null) {
                    writer.close();
                }
            } catch (IOException e) {
                System.out.println("Closing error: "
                        + e.getMessage());
            }
        }
    }
}
```



3.11 Conclusion

Try-with-resources is an effective method for managing resources in Java. It reduces code complexity, improves readability and automatically closes resources, helping prevent common resource-leakage problems.

Q4 – THREAD CREATION AND THE THREAD LIFE CYCLE

4.1 Introduction

Multithreading allows multiple tasks to execute concurrently within a single application.

In an examination department, four scanner stations may process answer scripts simultaneously. Each station can be represented as an independent thread.

Java provides two major methods for creating threads: extending the Thread class and implementing the Runnable interface.

4.2 Objectives

1. To understand multithreading.
2. To study the thread life cycle.
3. To create threads using Thread and Runnable.
4. To understand start(), sleep(), join() and isAlive().
5. To understand thread scheduling.
6. To compare different thread-creation approaches.

4.3 Thread Life Cycle

A Java thread passes through different states during its execution.

1. NEW: A thread object has been created but start() has not been called.

2. RUNNABLE: The thread is ready to execute or is executing.

3. BLOCKED: The thread is waiting to acquire a monitor lock.

4. WAITING: The thread is waiting indefinitely for another thread's action.

5. TIMED_WAITING: The thread is waiting for a specified period.

6. TERMINATED: The thread has completed its execution.

The main thread life-cycle flow is:

NEW → RUNNABLE → Execution → WAITING or TIMED_WAITING → RUNNABLE → TERMINATED.

Java represents both ready-to-run and actively running threads using the RUNNABLE state.

4.4 Methods of Thread Management

start(): Starts a new thread of execution.

run(): Contains the task performed by the thread.

sleep(): Temporarily pauses the current thread.

join(): Makes one thread wait for another thread to complete.

isAlive(): Checks whether a thread has started and has not yet terminated.

4.5 Thread vs Runnable

Extending Thread	Implementing Runnable
Extends Thread class	Implements Runnable interface
Cannot extend another class	Can extend another class
Task and thread are combined	Task and thread are separated
Less flexible	More flexible
Suitable for simple programs	Suitable for reusable tasks

Runnable is generally preferred because it separates the task from thread management.

4.6 Student A – Thread Design

Student A is responsible for explaining thread creation and the thread life cycle.

Why should start() be used instead of run()?

Calling start() creates a new thread of execution, and the JVM invokes run() on that thread.

Calling run() directly executes the method in the current thread and does not create a new thread.

Importance of join():

The join() method ensures that the main thread waits for other threads to finish.

In the scanner application, join() ensures that the final total is printed only after all four scanner stations have completed their tasks.

4.7 Student B – Behavioural Experiments

Student B is responsible for observing thread scheduling and execution order.

The program must be executed at least ten times, and the order in which scanner stations finish must be recorded.

An illustrative observation table is shown below.

Run	Possible finishing order
1	S1, S2, S3, S4
2	S2, S1, S4, S3
3	S3, S4, S1, S2
4	S1, S3, S2, S4
5	S4, S2, S3, S1
6	S2, S3, S1, S4
7	S3, S1, S4, S2
8	S4, S1, S2, S3
9	S1, S4, S3, S2
10	S2, S4, S1, S3

These are example orders, not measured results.

Thread priorities:

Java supports priorities from 1 to 10. Higher priority may influence scheduling, but it does not guarantee execution order.

Daemon threads:

Daemon threads execute background operations. They do not prevent the JVM from terminating after all user threads have completed.

4.8 Joint Conclusion

The same multithreaded program may produce different output orders because the JVM and operating system control thread scheduling.

A correct program should not depend on a particular execution order.

Synchronization, proper coordination and join() should be used wherever execution order affects correctness.

4.9 Alternative Concept and Complexity Analysis

Instead of manually creating four Thread objects, the scanner application can use ExecutorService with a fixed thread pool.

ExecutorService manages a group of reusable worker threads.

Let:

- n = number of answer scripts.
- p = number of worker threads.

For approximately equal independent tasks, the ideal processing time is proportional to n/p.

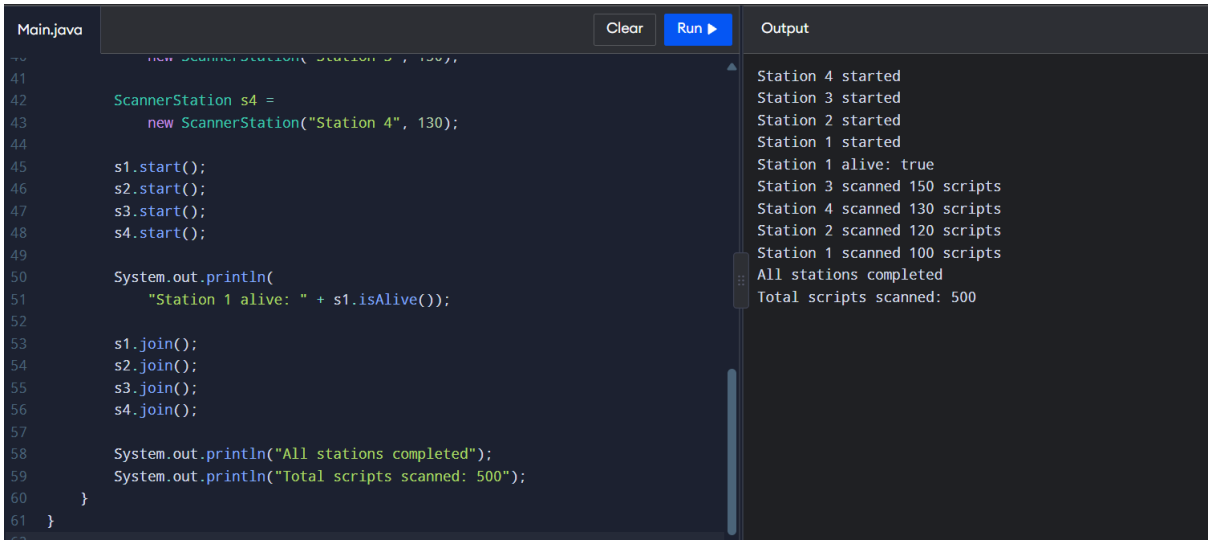
Approach	Expected processing time	Memory usage
Manual thread creation	Approximately O(n/p)	O(p) thread stacks
ExecutorService	Approximately O(n/p)	O(p) thread stacks plus task queue

The actual performance depends on task duration, scheduling overhead and available hardware resources.

PROGRAM

```
class ScannerStation extends Thread {
   String station;
   int scripts;
   ScannerStation(String station, int scripts) {
      this.station = station;
      this.scripts = scripts;
   }
   public void run() {
      System.out.println(station + " started");
      try {
         Thread.sleep(1000);
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
         return;
      }
      System.out.println(station +
            " scanned " + scripts + " scripts");
   }
}
public class Main {
```

```
  public static void main(String[] args)
          throws InterruptedException {
      ScannerStation s1 =
          new ScannerStation("Station 1", 100);
      ScannerStation s2 =
          new ScannerStation("Station 2", 120);
      ScannerStation s3 =
          new ScannerStation("Station 3", 150);
      ScannerStation s4 =
          new ScannerStation("Station 4", 130);
      s1.start();
      s2.start();
      s3.start();
      s4.start();
      System.out.println(
          "Station 1 alive: " + s1.isAlive());
      s1.join();
      s2.join();
      s3.join();
      s4.join();
      System.out.println("All stations completed");
      System.out.println("Total scripts scanned: 500");
  }
}
```



4.10 Conclusion

Multithreading allows examination scanner stations to process answer scripts concurrently. Thread creation, thread life-cycle management and ExecutorService improve application efficiency and provide better control over concurrent tasks.

Q5 – RACE CONDITIONS IN A CINEMA SEAT-BOOKING SYSTEM

5.1 Introduction

A cinema booking application allows multiple customers to reserve seats simultaneously.

A race condition occurs when multiple threads access shared data concurrently and the final result depends on the timing and order of their execution.

If two users attempt to book the same seat at the same time, both may identify the seat as available and complete the booking operation.

This creates a double-booking problem.

Java provides synchronization to prevent multiple threads from accessing a critical section simultaneously using the same lock.

5.2 Objectives

1. To understand race conditions.
2. To identify critical sections.
3. To understand synchronization.
4. To prevent double booking.
5. To compare synchronized methods and synchronized blocks.
6. To understand intrinsic locks.

5.3 Race Condition

A race condition occurs when multiple threads access and modify shared data without proper coordination.

In the cinema booking application, the shared data is the seat-booking array.

The booking operation contains three important steps:

1. Check whether the seat is available.
2. Mark the seat as booked.
3. Increase the booking count.

If these operations are not performed atomically, two threads may book the same seat.

5.4 Critical Section

The critical section is the part of the program that accesses shared data and must be protected.

In the cinema booking system, the check-and-book operation is the critical section.

Synchronization ensures that only one thread at a time can execute the protected operation on the same object.

5.5 Synchronization Strategies

1. Synchronized method

A synchronized method locks the current object before executing the method.

It is simple and suitable when the entire method requires protection.

2. Synchronized block

A synchronized block protects only the statements that access shared data.

It can improve concurrency when the remaining method operations do not require locking.

3. Static synchronization

A static synchronized method locks the Class object instead of an individual instance.

It is useful when multiple objects must coordinate access to shared static data.

4. Intrinsic lock

Every Java object has an intrinsic monitor lock.

Synchronized methods and blocks use these locks to coordinate access between threads.

5.6 Student A – Producing and Explaining the Race

Student A is responsible for reproducing and analysing the race condition.

The experiment requires:

1. Removing synchronization.
2. Creating multiple threads competing for the same seats.
3. Executing the application repeatedly.
4. Recording the number of successful bookings.
5. Comparing the results with the synchronized version.

Without synchronization, the number of bookings may exceed the actual number of available seats.

However, double booking is not guaranteed in every execution because it depends on thread scheduling.

5.7 Student B – Comparing Synchronization Strategies

Student B is responsible for comparing synchronization approaches and measuring their performance.

Strategy	Advantages	Disadvantages
Unsynchronized	Low locking overhead	Double booking possible
Synchronized method	Simple and reliable	Locks the entire method
Synchronized block	Protects only the critical section	Requires careful lock selection
Static synchronized	Protects shared static data	Can restrict concurrency across instances

The execution time for 10,000 bookings should be measured for each version.

The results must be recorded and compared.

5.8 Joint Engineering Decision

For a cinema containing 20 screens, a per-screen synchronization strategy is suitable.

Each screen should maintain its own booking information and lock.

This allows independent screens to process bookings concurrently while preventing multiple customers from reserving the same seat.

For a real production application, database transactions and unique seat-booking constraints should also be used because Java synchronization alone cannot protect against requests handled by multiple application servers.

5.9 Alternative Concept and Complexity Analysis

Other approaches can be used to prevent double booking.

AtomicBoolean:

AtomicBoolean supports atomic operations such as compareAndSet(). It can change a seat from available to booked without requiring a synchronized method.

ReentrantLock:

ReentrantLock provides explicit locking and unlocking operations.

ConcurrentHashMap:

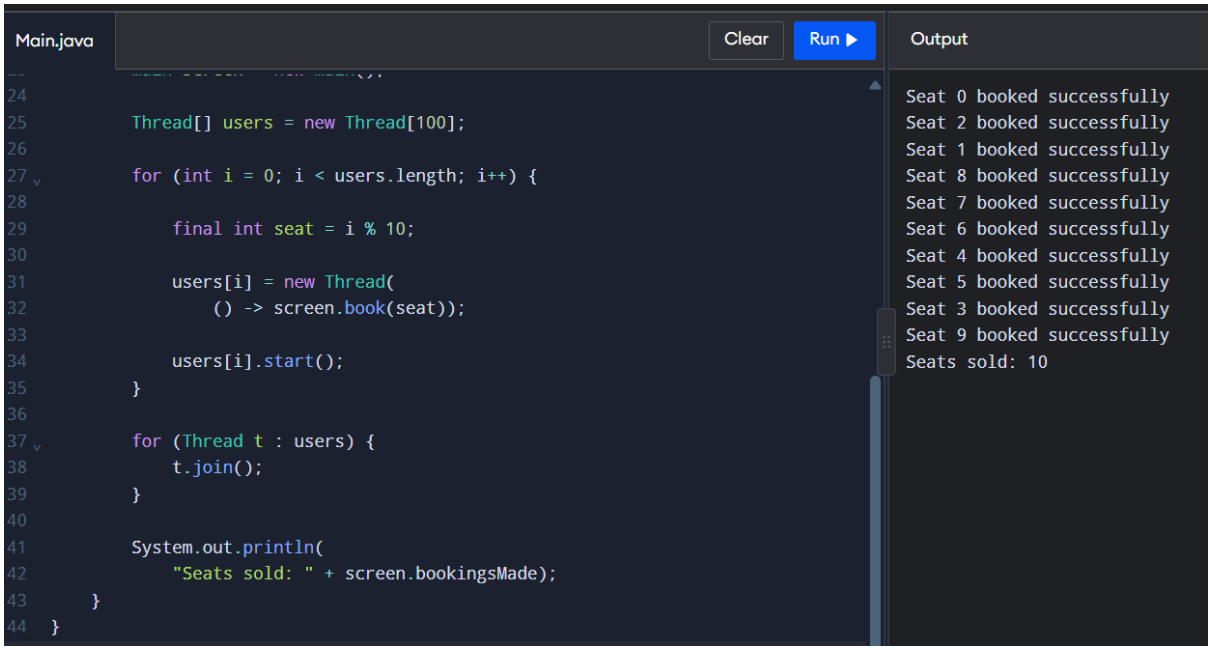
ConcurrentHashMap supports thread-safe operations such as putIfAbsent(), which can prevent duplicate seat insertion.

Approach	Time per booking	Additional space
Synchronized	O(1) basic operation	O(s) seat storage
AtomicBoolean[]	O(1) expected	O(s) atomic objects
ReentrantLock	O(1) basic operation, excluding contention	O(1) per lock scope
ConcurrentHashMap	Expected O(1)	O(s) entries

Here, s represents the total number of seats.

PROGRAM:

```
public class Main {
    private final boolean[] booked = new boolean[10];
    private int bookingsMade = 0;
    public synchronized void book(int seat) {
        if (!booked[seat]) {
            booked[seat] = true;
            bookingsMade++;
            System.out.println(
                "Seat " + seat + " booked successfully");
        }
    }
    public static void main(String[] args)
            throws InterruptedException {
        Main screen = new Main();
        Thread[] users = new Thread[100];
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10;
            users[i] = new Thread(
                () -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) {
            t.join();
        }
        System.out.println(
            "Seats sold: " + screen.bookingsMade);
    }
}
```



5.10 Conclusion

Synchronization is essential for maintaining data consistency in multithreaded applications. The cinema seat-booking system demonstrates how synchronized methods and blocks prevent race conditions and ensure that a seat cannot be sold more than once.

Q6 – DEADLOCK AND INTER-THREAD COMMUNICATION

6.1 Introduction

Multithreading applications often require multiple threads to share resources and communicate with one another.

Two important concepts are deadlock and inter-thread communication.

Deadlock occurs when two or more threads wait indefinitely for resources held by one another.

Inter-thread communication allows threads to coordinate their activities using wait(), notify() and notifyAll().

This question contains two parts:

- Part A: Hostel wallet transfers and deadlock prevention.
- Part B: Producer-consumer problem using a mess token counter.

6.2 Objectives

1. To understand deadlock.
2. To identify the four necessary conditions for deadlock.

3. To prevent deadlock using lock ordering.
4. To understand producer-consumer communication.
5. To implement wait(), notify() and notifyAll().
6. To differentiate sleep() and wait().

6.3 Part A – Deadlock in Hostel Wallet Transfers

A hostel wallet system allows students to transfer money between their accounts.

During a transfer, two wallet objects must be locked to prevent inconsistent balance updates.

If two threads acquire these locks in opposite orders, a deadlock may occur.

Four Conditions of Deadlock

1. Mutual Exclusion

Only one thread can hold a particular lock at a time.

2. Hold and Wait

A thread holds one resource while waiting for another resource.

3. No Preemption

A lock cannot be forcibly taken away from the thread holding it.

4. Circular Wait

Two or more threads form a circular chain of waiting.

All four conditions must exist simultaneously for the traditional deadlock scenario.

6.4 Deadlock Prevention

Deadlock can be prevented by using a consistent lock-ordering rule.

For example, wallet objects can be locked according to their unique IDs.

The wallet with the smaller ID must always be locked first, followed by the wallet with the larger ID.

This prevents circular waiting and reduces the possibility of deadlock.

6.5 Part B – Producer-Consumer Problem

The hostel mess counter stores a maximum of five meal tokens.

The cook acts as the producer and creates tokens.

Students act as consumers and collect tokens from the counter.

The producer must wait when the buffer is full, and consumers must wait when the buffer is empty.

Java provides wait(), notify() and notifyAll() to coordinate these operations.

6.6 Inter-Thread Communication

wait():

The wait() method makes the current thread wait until another thread performs a notification or another permitted wakeup occurs.

It releases the monitor of the object on which it is called.

notify():

The notify() method wakes one waiting thread on the same object's monitor.

notifyAll():

The notifyAll() method wakes all threads waiting on the same object's monitor.

The awakened threads must compete to reacquire the monitor.

6.7 Student A – Producer-Consumer Design

Student A is responsible for designing the producer-consumer system.

Why must wait() be called inside a loop?

A thread may wake up without the required condition becoming true.

This may happen because of a spurious wakeup or because another thread changes the shared data before the awakened thread reacquires the lock.

Therefore, the condition must be checked repeatedly using a while loop.

Why must wait() be called inside synchronized?

The thread must own the object's monitor before calling wait().

Otherwise, Java throws IllegalMonitorStateException.

Difference between notify() and notifyAll():

notify() wakes one waiting thread, whereas notifyAll() wakes all waiting threads.

notifyAll() is useful when multiple consumers and producers may be waiting for different conditions.

Difference between sleep() and wait():

sleep()	**wait()**
Method of Thread class	Method of Object class
Does not release an owned monitor	Releases the monitor on which wait() is called
Used for timed pauses	Used for inter-thread communication
Can be called without synchronization	Requires ownership of the object's monitor

6.8 Student B – Deadlock Analysis and Prevention

Student B is responsible for constructing and analysing a deadlock situation.

A possible situation is:

- Thread 1 locks Wallet A and waits for Wallet B.
- Thread 2 locks Wallet B and waits for Wallet A.

Both threads remain blocked because each waits for a resource held by the other.

The four necessary conditions are mutual exclusion, hold and wait, no preemption and circular wait.

Deadlock can be prevented by consistently acquiring wallet locks in increasing ID order.

6.9 Joint Conclusion

Deadlock and producer-consumer problems both involve coordination between multiple threads and shared resources.

Deadlock prevention ensures that threads do not remain blocked indefinitely.

Producer-consumer communication ensures that threads operate safely without exceeding buffer capacity or consuming unavailable items.

Both concepts are important for developing reliable multithreaded applications.

6.10 Alternative Concept and Complexity Analysis

ArrayBlockingQueue:

ArrayBlockingQueue is a predefined Java concurrent collection that provides a bounded blocking queue.

It automatically manages thread synchronization and waiting.

For the mess counter, an ArrayBlockingQueue with a capacity of five can replace manual wait() and notifyAll() operations.

ReentrantLock with tryLock():

ReentrantLock supports tryLock() to attempt lock acquisition without waiting indefinitely.

It can also use a timeout to prevent threads from waiting forever for a lock acquisition.

Operation	**Time complexity**	**Space complexity**
Put token	O(1) basic operation	O(5) buffer
Take token	O(1) basic operation	O(5) buffer
Wallet transfer	O(1) basic operation	O(1) additional lock space per transfer
ArrayBlockingQueue put/take	O(1) basic operation	O(5) queue storage

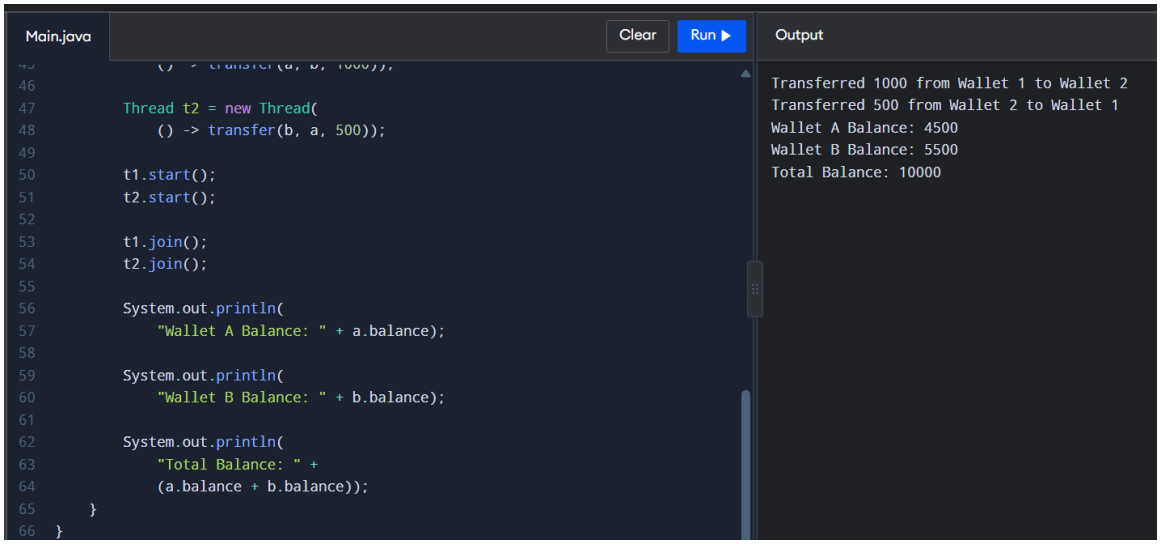
The actual execution time depends on thread contention and waiting time.

ArrayBlockingQueue is generally less error-prone because it internally manages synchronization and thread coordination.

PROGRAM:

```
class Wallet {
   int balance;
   final int id;
   Wallet(int id, int balance) {
      this.id = id;
      this.balance = balance;
   }
}
```

```
public class Main {
    static void transfer(Wallet a, Wallet b, int amount) {
        Wallet first = a.id < b.id ? a : b;
        Wallet second = a.id < b.id ? b : a;
        synchronized (first) {
            synchronized (second) {
                if (a.balance >= amount) {
                    a.balance -= amount;
                    b.balance += amount;
                    System.out.println(
                        "Transferred " + amount +
                        " from Wallet " + a.id +
                        " to Wallet " + b.id);
                }
            }
        }
    }
    public static void main(String[] args)
            throws InterruptedException {
        Wallet a = new Wallet(1, 5000);
        Wallet b = new Wallet(2, 5000);
        Thread t1 = new Thread(
            () -> transfer(a, b, 1000));
        Thread t2 = new Thread(
            () -> transfer(b, a, 500));
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println(
            "Wallet A Balance: " + a.balance);
        System.out.println(
            "Wallet B Balance: " + b.balance);
        System.out.println(
            "Total Balance: " +
            (a.balance + b.balance));
    }
}
```



6.11 Conclusion

This question demonstrates deadlock prevention and inter-thread communication in Java. Consistent lock ordering prevents circular waiting, while wait() and notifyAll() coordinate producer and consumer threads safely.

Q7 – DESIGN A CONCURRENT, FAULT-TOLERANT PROCESSOR FROM FIRST PRINCIPLES

7.1 Introduction

The examination department must process approximately 10,000 answer-script score files at the end of each semester.

Each file must be read, validated and processed to calculate the student's result.

However, some files may be corrupted or missing.

The system must process all valid files efficiently, handle individual failures without stopping the entire application and generate a final summary of successful and failed files.

Java provides ExecutorService, Callable, Future and concurrent collections to develop such applications.

7.2 Objectives

1. To design a concurrent processing system.
2. To understand ExecutorService.
3. To implement Callable and Future.
4. To understand concurrent collections.
5. To process multiple files simultaneously.

6. To handle corrupted and missing files.
7. To measure execution time.
8. To generate an accurate final summary.

7.3 Stage 1 – Interpret the Problem

A. Tasks That Can Run in Parallel

The following operations can be executed independently:

1. Reading student score files.
2. Validating student marks.
3. Calculating total marks.
4. Calculating average marks.
5. Determining pass or fail status.
6. Recording successful results.
7. Recording failed files.

Each student file can be processed independently by a Callable task.

B. Shared Data

The system maintains shared information, including:

- Successful processing count.
- Failed processing count.
- Student result information.
- Failure details.
- Total execution time.

Concurrent collections are used to store shared results safely.

C. Failure Identification

Failure	**Cause**	**Handling**
FileNotFoundException	Missing file	Record failure
IOException	File-reading problem	Record failure
NumberFormatException	Invalid marks	Record failure
Invalid marks	Marks outside permitted range	Record failure
ExecutionException	Task execution failure	Record failure
InterruptedException	Main thread interrupted	Restore interrupt status

D. Measurable Requirements

1. The system must process 10,000 files.
2. Each file must be processed independently.
3. One failed file must not stop other tasks.
4. Every file must be counted as successful or failed.
5. The final summary must contain accurate counts.
6. The system must support thread pools of 1, 2, 4 and 8 threads.
7. The execution time must be recorded.

7.4 Stage 2 – Design (Student A)

A. System Architecture

The proposed system consists of the following stages:

1. Input score files.
2. File processor.
3. ExecutorService.
4. Callable tasks.
5. Future results.
6. Validation layer.
7. Concurrent collections.
8. Final summary.

System workflow:

Input Files → File Processor → ExecutorService → Callable Tasks → Future Results → Validation → Success and Failure Collections → Final Summary.

B. System Components

Component	**Responsibility**
FileProcessor	Coordinates file processing
ScoreTask	Reads and validates one file
ExecutorService	Manages worker threads
Callable	Performs processing and returns results
Future	Retrieves task results
ConcurrentHashMap	Stores successful results
ConcurrentLinkedQueue	Stores failure information

C. Exception Handling Design

Each Callable task processes one file independently.

If a file is missing, corrupted or contains invalid marks, the corresponding task records the failure.

Other tasks continue processing without being interrupted by that expected file-level failure.

Unexpected task failures can be detected using Future.get(), which may throw ExecutionException.

D. Pseudocode

START

1. Read all input file names.
2. Create an ExecutorService with a fixed thread pool.
3. Create a concurrent collection for successful results.
4. Create a concurrent collection for failed records.
5. Submit a Callable task for every file.
6. Read and validate the file inside each task.
7. Calculate the student's result.
8. Record successful or failed processing.
9. Retrieve the Future results.
10. Shut down the executor.
11. Calculate execution time.
12. Display the final summary.

END.

7.5 Stage 3 – Implement and Measure (Student B)

Student B is responsible for implementing the concurrent processing system.

The implementation must:

1. Generate or prepare input score files.
2. Inject corrupted files.
3. Include missing files.
4. Process the files using ExecutorService.
5. Use Callable and Future to obtain results.
6. Store successful and failed results in concurrent collections.
7. Measure execution time.
8. Verify the final summary.

A. Thread Pool Experiment

The assignment requires the program to be executed using four different thread pool sizes.

Thread pool size	Total files	Successful files	Failed files	Execution time
1	10,000	To be measured	To be measured	To be measured
2	10,000	To be measured	To be measured	To be measured
4	10,000	To be measured	To be measured	To be measured
8	10,000	To be measured	To be measured	To be measured

The actual execution time must be recorded after running the program.

B. Small Hand-Checked Test

A small dataset can be used to verify the processing results.

File	Input condition	Expected result
student1.txt	Valid marks	Success
student2.txt	Valid marks	Success
student3.txt	Alphabetic data	Failure
student4.txt	Missing file	Failure

Expected summary:

- Total files: 4.
- Successful files: 2.
- Failed files: 2.

C. Failure Handling

The application must handle individual file failures without stopping the entire processing operation.

Missing files, corrupted data and invalid marks must be recorded separately.

7.6 Stage 4 – Defend the Design (Joint)

A. Recommended Thread Pool Size

A four-thread pool is a reasonable starting point for the experiment.

However, the best thread pool size depends on:

- Processor capacity.
- Number of CPU cores.
- Disk speed.
- File sizes.
- Memory availability.
- Nature of the processing tasks.

More threads do not always improve performance.

Excessive threads may increase context switching, memory consumption, disk contention and scheduling overhead.

B. How Does the System Guarantee a Complete Summary?

The application submits one task for every input file.

It collects the results and classifies every task as successful or failed.

The final summary must satisfy:

Total Files = Successful Files + Failed Files.

For example:

10,000 = 9,998 + 2.

The final summary should be displayed only after all submitted tasks have completed.

C. Remaining Limitation

One limitation is that storing all Future objects and result information in memory may consume considerable memory for extremely large datasets.

A scalable implementation can use bounded task submission and process completed tasks incrementally.

7.7 Alternative Concept and Complexity Analysis

A. Parallel Streams

Parallel streams allow collections of files to be processed concurrently.

They provide a simpler approach to parallel execution and reduce the amount of manual thread-management code.

However, they provide less direct control over thread-pool configuration, and exception handling must be designed carefully.

B. One Thread Per File

Another approach is to create a separate thread for every file.

For 10,000 files, this would create 10,000 threads.

This approach is generally inefficient because it increases memory consumption and thread-scheduling overhead.

C. Complexity Comparison

Let:

- n = number of files.
- p = number of worker threads.
- s = average processing size of each file.

Approach	Expected processing time	Memory usage	Failure handling
ExecutorService	Approximately O(ns/p) under ideal conditions	O(p) thread stacks plus queued tasks/results	Individual task failures can be isolated
Parallel streams	Approximately O(ns/p) under ideal conditions	Managed worker threads and intermediate data	Unhandled exceptions may affect the operation
One thread per file	Approximately O(s) ideal parallel work, with high overhead	O(n) thread stacks	Must handle failures separately in each thread

Actual performance depends on task distribution, disk I/O, available processors, memory and scheduling overhead.

PROGRAM:

```
import java.util.*;
import java.util.concurrent.*;
class Result {
   String fileName;
   boolean success;
```

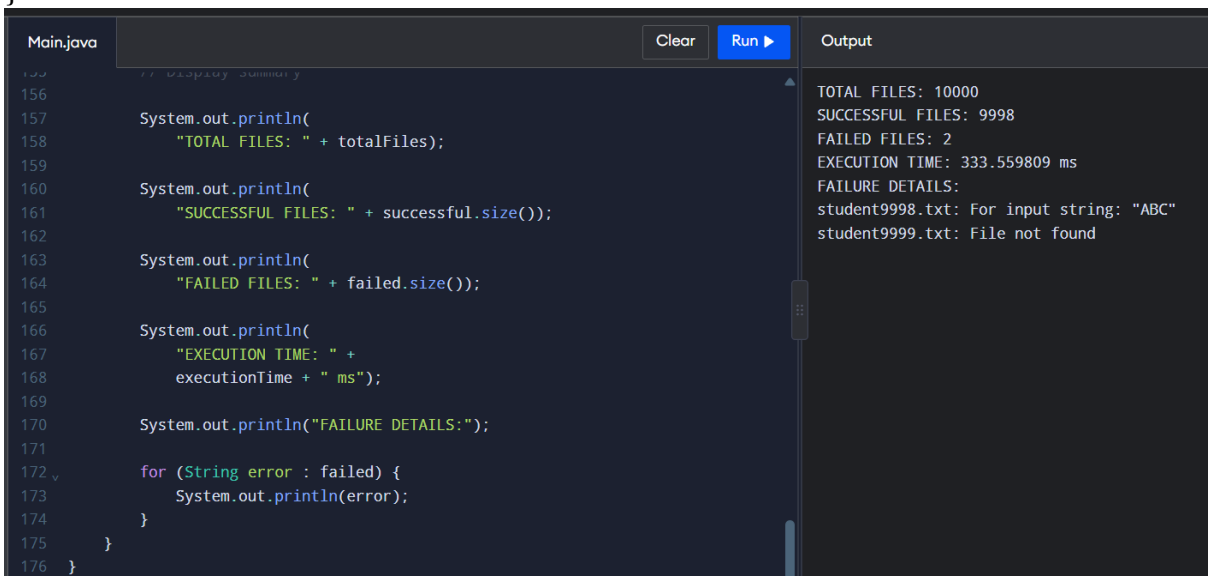
```
    double average;
    String message;
    Result(String fileName, boolean success,
           double average, String message) {
        this.fileName = fileName;
        this.success = success;
        this.average = average;
        this.message = message;
    }
}
class ScoreTask implements Callable<Result> {
    private final String fileName;
    private final String data;
    ScoreTask(String fileName, String data) {
        this.fileName = fileName;
        this.data = data;
    }
    public Result call() {
        try {
            if (data == null) {
                throw new IllegalArgumentException(
                    "File not found");
            }
            String[] marks = data.split(",");
            int total = 0;
            for (String value : marks) {
                int mark = Integer.parseInt(value.trim());

                if (mark < 0 || mark > 100) {
                    throw new IllegalArgumentException(
                        "Invalid mark: " + mark);
                }
                total += mark;
            }
            double average = (double) total / marks.length;
            return new Result(
                fileName, true, average,
                "Processed successfully");
        } catch (Exception e) {
            return new Result(
                fileName, false, 0, e.getMessage());
        }
    }
}
public class Main {
    public static void main(String[] args)
            throws InterruptedException {
        int totalFiles = 10000;
        int poolSize = 4;
```

```
ExecutorService executor =
    Executors.newFixedThreadPool(poolSize);
 ConcurrentHashMap<String, Double> successful =
    new ConcurrentHashMap<>();
 ConcurrentLinkedQueue<String> failed =
    new ConcurrentLinkedQueue<>();
 List<Future<Result>> futures =
    new ArrayList<>();
 long startTime = System.nanoTime();
 // Submit 10000 student tasks
 for (int i = 1; i <= totalFiles; i++) {
    String fileName = "student" + i + ".txt";
    String data;
    if (i == 9999) {
       data = null;          // Missing file
    } else if (i == 9998) {
       data = "80,ABC,90";   // Corrupted file
    } else {
       data = "80,90,85";    // Valid file
    }
    futures.add(
       executor.submit(
          new ScoreTask(fileName, data)));
 }
 // Collect task results
 for (Future<Result> future : futures) {
    try {
       Result result = future.get();
       if (result.success) {
         successful.put(
             result.fileName,
             result.average);
       } else {
          failed.add(
             result.fileName + ": " +
             result.message);
       }
    } catch (ExecutionException e) {

       failed.add(
          "Unexpected failure: " +
          e.getCause());
    }
 }
 executor.shutdown();
 executor.awaitTermination(
    1, TimeUnit.MINUTES);
 long endTime = System.nanoTime();
double executionTime =
    (endTime - startTime) / 1_000_000.0;
```

```
        // Display summary
        System.out.println(
            "TOTAL FILES: " + totalFiles);
        System.out.println(
            "SUCCESSFUL FILES: " + successful.size());
        System.out.println(
            "FAILED FILES: " + failed.size());
        System.out.println(
            "EXECUTION TIME: " +
            executionTime + " ms");
        System.out.println("FAILURE DETAILS:");
        for (String error : failed) {
            System.out.println(error);
        }
    }
}
```



7.8 Conclusion

The concurrent result-processing system demonstrates how Java concurrency features can be used to develop reliable and fault-tolerant applications.

ExecutorService manages worker threads, Callable performs independent tasks, Future retrieves task results, and concurrent collections store shared processing information.

The system can process multiple files concurrently, handle individual failures and generate a final summary without stopping the entire application because of one corrupted or missing file.

CONCLUSION

This assignment provided practical knowledge of exception handling and multithreading in Java.

The seven questions demonstrated how Java concepts can be applied to real-world applications.

The ATM simulation explained exception handling and failure classification. The online examination portal demonstrated user-defined exceptions, propagation and chaining. The sales report generator explained resource management and try-with-resources.

The scanner station application demonstrated thread creation and the thread life cycle. The cinema seat-booking system explained race conditions and synchronization. The hostel wallet and mess token applications demonstrated deadlock prevention and inter-thread communication.

Finally, the semester result-processing application demonstrated concurrent programming using ExecutorService, Callable, Future and concurrent collections.

Overall, the assignment improved understanding of Java programming, exception handling, multithreading, debugging, concurrency and software reliability.